AI Agent Creation: Full Architecture Guide with Agentic Data Fetching & LLM Integration

Oct 29, 2026 · @Arffy technologies

Introduction: What is an AI Agent?

An AI agent is fundamentally different from a one-shot LLM call. An AI agent architecture is the structural design that lets an LLM observe its environment, reason about a goal, and act on it by calling tools in a repeating loop. Unlike traditional chatbots that generate a response and stop, agents continuously reason, make decisions, and take actions until they complete a task.

Core Characteristics of Agentic Systems

Three properties separate an agentic system from a simple prompt:

1. **Perception** — The agent observes its environment continuously (user input, tool outputs, file content, API responses, sensor data)
2. **Reasoning** — The LLM analyzes the situation, plans next steps, and decides which action to take
3. **Action** — The agent executes tools, APIs, or functions and feeds results back into the reasoning loop

The architectural choices that make agents work in production involve how the loop is shaped, what the agent is allowed to remember, which tools it sees, and how it is verified, audited, and stopped.

Why Agentic Architecture Matters

Single-pass LLM calls are unreliable at scale. Agents enable:

- **Multi-step reasoning**: Breaking complex tasks into sequential decisions
- **Adaptive retrieval**: Pulling fresh data exactly when needed, not upfront
- **Error correction**: Evaluating results and retrying with different approaches
- **Tool integration**: Extending the LLM with APIs, databases, search engines, and domain-specific functions

- **Knowledge management**: Combining real-time data with long-term memory and learned patterns

Core Architecture: Four-Layer Model

Production agents follow a four-layer architecture that transforms language models into autonomous systems:

Layer 1: Reasoning Layer (LLM Brain)

This is the decision-making core. The LLM:

- Receives context: user input, tool outputs, system instructions, and memory
- Reasons about what needs to be done next
- Decides which tools to call, if any
- Generates structured function calls with parameters
- Evaluates outcomes and adapts strategy

The reasoning layer determines the agent's intelligence, creativity, and ability to handle novel situations.

Layer 2: Orchestration Layer (Control Flow)

This layer bridges the LLM and external world:

- Receives function call requests from the reasoning layer
- Validates and executes those calls
- Formats results back into natural language/structured data
- Manages the loop: feeding outputs back to the LLM for the next reasoning step
- Handles errors, retries, and fallbacks
- Tracks state and decision history

The orchestration layer is responsible for reliability—ensuring the agent actually executes what it decides.

Layer 3: Memory and Data Layer

This stores and retrieves information the agent needs:

- **Short-term context**: Recent messages, current task state, conversation history (stored in prompt)

- **Long-term memory**: Learned facts, user preferences, past decisions (vector databases, embeddings)
- **Episodic memory**: Specific events with timestamps, outcomes, and what was learned
- **External data access**: Real-time APIs, databases, knowledge graphs
- **RAG systems**: Retrieval-augmented generation to ground responses in fresh data

Without proper data access, agents generate hallucinations. With it, they ground reasoning in facts.

Layer 4: Tool Integration Layer (Action Capabilities)

This defines what the agent can actually do:

- **Read tools**: Search engines, database queries, document retrieval, API data fetching
- **Write tools**: Creating records, updating systems, sending messages
- **Transform tools**: Summarization, classification, translation, data formatting
- **Reasoning tools**: Calculators, logic engines, constraint solvers
- **Compound tools**: Higher-level abstractions combining multiple capabilities

Tools are where potential becomes action. Well-designed tools with clear schemas enable precise agent behavior.

The Agentic Loop: Perception-Reasoning-Action

The agentic loop is the heartbeat of every AI agent. It repeats continuously until the agent completes its task.

Step 1: Perception (Observe)

The agent receives input:

- User messages or task specifications
- Outputs from previous tool calls
- File content or uploaded documents
- API responses and data fetches
- Sensor data or environment state
- System messages and feedback

All of this forms the raw context that the LLM will reason about in the next step.

Step 2: Reasoning (Think)

The LLM processes the perceived information:

1. **Understand the current state**: What information do I have? What's the goal? What's already been tried?
2. **Plan next action**: What should I do to make progress? Do I need more information?
3. **Decide which tool(s) to call**: If I need external capabilities, which tool(s) are best?
4. **Generate structured calls**: Output function calls with proper parameters
5. **Consider alternatives**: Are there better approaches? What could go wrong?

Depending on the LLM's capabilities, it may specify multiple function calls to run in parallel before the next reasoning step.

Step 3: Action (Execute)

The orchestration layer executes what the LLM decided:

1. **Validate the request**: Are the parameters valid? Are there constraints to check?
2. **Execute the tool**: Call the API, query the database, run the function
3. **Capture the result**: Return data, error message, or status
4. **Format for the next iteration**: Convert the result into natural language or structured data

Step 4: Loop Back (Iterate)

The result becomes the perception for the next iteration:

- If the agent got useful information, it reasons about what to do next
- If an error occurred, it reasons about recovery or alternatives
- If the goal is met, it can generate a final response and exit
- If more steps are needed, the loop continues

Example Loop in Action

User: "Find me flights to Paris next week under $500."

1. **Perception**: Agent receives the request
2. **Reasoning**: "I need to search for flights. I should call a flight search tool with Paris as destination and next week's dates."

3. **Action**: Calls `search_flights(destination="Paris", start_date="2026-11-05", end_date="2026-11-12", max_price=500)`
4. **Perception**: Receives list of 15 flights ranging from $480-$3,000
5. **Reasoning**: "I need to filter these by price and present only the cheap options. I'll also fetch details on the cheapest ones."
6. **Action**: Calls `get_flight_details(flight_id=12345)` for the top 3 budget flights
7. **Perception**: Gets detailed info on baggage, seat selection, etc.
8. **Reasoning**: "I have enough information to answer the user's question."
9. **Output**: Returns the 3 best options under $500

Key Insight: The Loop Adapts

Unlike fixed workflows, the agentic loop is dynamic. The agent:

- Adjusts strategy based on what it learns
- Calls tools it didn't initially anticipate
- Stops early if the goal is met
- Retries with different approaches if something fails
- Can ask clarifying questions if the goal is ambiguous

Data Fetching & Real-Time Retrieval

The ability to fetch fresh data is what separates agents from pre-trained models. Agents must access external information on demand to stay current and accurate.

Data Fetching Mechanisms

APIs and External Services

- REST APIs for real-time data (weather, stock prices, flight status)
- GraphQL for efficient querying of complex data
- Webhooks for event-driven updates
- Rate limiting and timeout handling required

Databases and Data Systems

- Structured data: SQL queries to relational databases
- Unstructured: Document stores and file systems

- Time-series data: Stock prices, sensor readings, metrics
- Transactional systems: Order management, CRM, ERP

Search and Information Retrieval

- Web search engines for public information
- Internal search for proprietary documents
- Keyword-based and semantic search
- Real-time indexing for fresh content

Knowledge Systems

- Knowledge graphs: Entity relationships and facts
- Vector databases: Semantic similarity matching
- Document stores: PDFs, wikis, internal docs
- Structured knowledge bases: FAQs, reference tables

Agentic Data Fetching Pattern

Unlike traditional RAG which retrieves all needed data upfront, agents fetch adaptively:

1. **Initial reasoning with incomplete data**: Agent starts with what it has
2. **Identifies information gaps**: "I need to know X to proceed"
3. **Formulates targeted queries**: Specific API calls or search queries
4. **Evaluates results**: Are these the right answers? Do I need more?
5. **Refines requests iteratively**: Narrows down or explores tangents
6. **Stops when satisfied**: Enough data to answer the user's question

Common Data Fetching Failures

Hallucinated Data Hierarchies

If a tool schema doesn't enforce strict hierarchy, the LLM might confidently make up IDs. Solution: Tool definitions must guide the LLM through required sequences. Example: To create a task, the agent must first know the project ID, then workspace ID, then assignee ID.

Parallel Execution Chaos

An agent deciding to execute 50 tool calls in parallel might overload systems. Solution: Implement step budgets, rate limiting, and sequential execution modes for sensitive operations.

Stale Data Integration

Older cached data mixed with fresh queries leads to contradictions. Solution: Timestamp all fetched data, mark staleness explicitly, and refresh critical data on every loop.

Best Practices for Robust Data Fetching

- **Tool schema design**: Clear parameters, constraints, required fields
- **Error handling**: Graceful degradation, fallback sources, retry logic
- **Data validation**: Check types, ranges, and logical consistency
- **Observability**: Log what data was fetched, when, and from where
- **Cost control**: Monitor API calls, implement quotas for expensive operations
- **Latency optimization**: Batch requests, cache frequently accessed data, use parallel calls when safe

Tool Integration & Function Calling

Tools are how agents take action. Modern LLMs have native function calling: instead of parsing freeform text, the model returns structured tool calls that the system executes.

The Tool Use Pattern

This is the foundational pattern for all agentic systems:

1. **Tool Definition**: You provide the LLM with a menu of available tools
    - Clear name and description
    - Input parameters with types, constraints, and examples
    - Output format specification
2. **LLM Decision**: The LLM analyzes the user's request and decides if it needs outside information or actions
    - If it can answer directly, it does
    - If it needs external data or capabilities, it decides which tool(s) to use
3. **Function Call Generation**: Instead of a normal sentence, the LLM outputs a structured request (usually JSON) containing:

- Tool name
- Argument values with proper types
- Any metadata (e.g., timeout, retry count)

4. **Tool Execution**: Your application catches the call and executes it
   - Validates parameters
   - Calls the actual API/function/code
   - Gets the result
   - The LLM does NOT execute code itself
5. **Observation**: The result is fed back to the LLM as a new message
   - Success: "Here's what the database returned"
   - Failure: "This API call failed with error: invalid parameter"
6. **LLM Processing**: The LLM reads the observation and either:
   - Generates a final, human-friendly response for the user
   - Calls another tool to continue its reasoning
   - Asks a clarifying question

Tool Design Best Practices

Atomic Operations

Tools should do one thing well. Instead of one "update_customer" tool, have:

- `update_customer_name`
- `update_customer_email`
- `update_customer_address`

This makes each tool's purpose clear and prevents unintended side effects.

Strict Parameter Validation

```
{
  "name": "search_flights",
  "parameters": {
    "type": "object",
    "properties": {
      "destination": {
        "type": "string",
        "description": "IATA code (e.g., 'CDG' for Paris)"
      },
```

```
      "date": {
        "type": "string",
        "format": "date",
        "description": "YYYY-MM-DD format"
      },
      "max_price": {
        "type": "integer",
        "minimum": 0,
        "description": "Maximum price in USD"
      }
    },
    "required": ["destination", "date"]
  }
}
```

Injected Context

Don't ask the LLM to provide session IDs, API keys, or user identities. Inject these at runtime:

```
# Wrong: LLM must provide the session_id
get_user_history(session_id=???)

# Right: System injects it
get_user_history()  # session_id is automatically added
```

Informative Error Messages

When a tool fails, tell the LLM WHY:

```
# Bad
{"error": "failed"}

# Good
{"error": "Invalid destination code: 'Pairs'. Did you mean 'CDG'? Valid
codes: CDG, ORY, EGC"}
```

Chunked Results for Large Data

If a tool returns 1000 results, don't send all of them to the LLM:

```
{
  "results": [...first 10...],
  "total": 1000,
  "next_page": "search_flights(destination='CDG', page=2)"
}
```

Parallel vs. Sequential Execution

Parallel Execution

When the LLM specifies multiple independent tool calls, execute them simultaneously:

```
LLM decides:
  - search_flights(destination='CDG')
  - search_hotels(city='Paris')
  - get_weather(city='Paris')

System executes all 3 in parallel, then returns results to LLM
```

Benefit: Faster responses. Drawback: Higher resource usage, potential race conditions.

Sequential Execution

For dependent operations, execute in order:

```
Step 1: create_order() → returns order_id
Step 2: add_items_to_order(order_id=<from step 1>)
Step 3: checkout_order(order_id=<from step 1>)
```

Native Tool Calling vs. Text-Based

Native Tool Calling (Preferred)

- LLM returns structured JSON tool calls
- No parsing errors
- Type-safe
- Self-correcting
- Supported by all modern LLMs (Claude, GPT-4, Gemini, Llama 2+)

Text-Based Tool Calling (Avoid if possible)

- LLM embeds tool calls in prose text
- Requires regex/NLP parsing

- Prone to hallucination
- Hard to validate
- Only use for legacy systems

Memory Systems: Storing & Retrieving Knowledge

Agents need different types of memory to function effectively. Modern agents combine multiple memory layers.

Short-Term Memory (Session Context)

This is the recent history available to the current reasoning step:

- **Recent messages**: The last N turns of conversation
- **Current task state**: What's being worked on, partial results
- **Conversation history**: Full thread from start of session
- **Variables**: Computed values, IDs, extracted information

Stored in the prompt itself, it grows larger with longer conversations. Cost scales linearly with context.

Long-Term Memory (Persistent Knowledge)

This persists across conversations and sessions:

Semantic Memory

- Facts and concepts the agent learned
- Relationships between entities
- Domain knowledge
- Rules and constraints
- Stored in vector databases with semantic search

Episodic Memory

- Specific past events with timestamps
- What happened, when, and outcomes
- User preferences learned over time
- Previous decisions and their results
- Usually stored in structured databases or logs

Procedural Memory

- How to do things
- Scripts for common tasks
- Learned workflow optimizations
- Stored as prompts, tool configurations, or decision trees

Vector Databases & Semantic Search

Vector databases power modern long-term memory:

1. **Embedding**: Convert text into dense vectors (embeddings)
   - "Paris is the capital of France" → [0.23, -0.51, 0.88, ...]
   - Semantically similar texts map to nearby points
2. **Storage**: Store vectors alongside original content and metadata

```
{
  "id": "doc_123",
  "embedding": [0.23, -0.51, 0.88, ...],
  "text": "Paris is the capital of France",
  "source": "wikipedia",
  "timestamp": "2026-10-29"
}
```

3. **Retrieval**: Find similar documents via semantic search
   - Agent query: "What's the most important city in France?"
   - Embedded query: [0.22, -0.49, 0.89, ...] (similar to Paris doc)
   - Returns top-k similar vectors
4. **Context Injection**: Retrieved chunks injected into the prompt

```
User: "Tell me about the capital of France"
[Retrieved]: "Paris is the capital of France. It's known for the Eiffel
Tower..."
LLM prompt: "Based on the following context: ... answer the question"
```

Popular vector databases: Pinecone, Chroma, Qdrant, Milvus, Weaviate, FAISS.

Memory Challenges & Solutions

Stale or Conflicting Information

Problem: Agents retrieve outdated facts that contradict fresh data.

Solution:

- Timestamp all stored knowledge
- Prioritize recent data in retrieval
- Verify retrieved facts with fresh API calls when critical
- Mark confidence/currency levels on stored facts

Memory Explosion

Problem: Context window fills up as conversations grow.

Solution:

- Summarize old conversations into key facts
- Use hierarchical memory (summary + recent + relevant)
- Implement sliding window for short-term memory
- Move old messages to long-term storage

Irrelevant Retrieval

Problem: Semantic search returns information that's technically similar but contextually wrong.

Solution:

- Use hybrid search (keyword + semantic)
- Filter by metadata before semantic search
- Rerank results using more sophisticated models
- Provide query-specific examples to the embedding model

Privacy and Data Isolation

Problem: Multi-user agents might leak information between users.

Solution:

- Namespace memory by user or session
- Filter retrieval by access permissions
- Encrypt sensitive data at rest
- Audit what each agent accesses

Retrieval-Augmented Generation (RAG)

RAG grounds LLM responses in external knowledge. Instead of relying only on what's in the LLM's training data, RAG retrieves relevant documents and injects them into the prompt before generation.

Traditional RAG Pipeline

Traditional RAG follows a simple retrieve-then-generate pattern:

1. **User asks a question**
2. **Retrieve relevant documents** via semantic search
3. **Inject retrieved chunks** into the prompt
4. **Generate answer** conditioned on the retrieved context
5. **Return response** with sources cited

Problem: Single-shot retrieval often misses what you need. Complex questions requiring multi-hop reasoning or data from multiple sources often fail.

Agentic RAG (Advanced Pattern)

Agentic RAG upgrades traditional RAG by embedding an autonomous agent into the retrieval loop. Instead of one retrieval, the agent makes multiple intelligent retrieval decisions.

How Agentic RAG Works

The agent cycles through planning, retrieval, evaluation, and refinement:

1. **Understand the query**: Break it into sub-questions or identify information gaps
2. **Retrieve strategically**: Decide which specific data sources to query
3. **Evaluate results**: Are these the right answers? Sufficient for the task?
4. **Reason and refine**: Make connections between retrieved facts
5. **Iterate**: If more information is needed, retrieve again
6. **Generate final answer**: Synthesize all evidence into a response

Example: Financial Document Q&A

Question: "What was our revenue growth in Q3 2026 compared to the previous quarter?"

Single-pass RAG might retrieve marketing documents about Q3 goals instead of financial reports.

Agentic RAG:

1. **Reasoning**: "I need Q3 2026 and Q2 2026 financial figures. Let me search for Q3 report first."
2. **Retrieval 1**: Search for "Q3 2026 financial report" → finds earnings document
3. **Evaluation**: "Good, but I also need Q2 data for comparison."
4. **Retrieval 2**: Search for "Q2 2026 revenue" → finds Q2 report
5. **Reasoning**: "I have Q3 revenue: $500M, Q2: $450M. Growth = 11%. Let me verify this calculation."
6. **Retrieval 3** (optional): "Let me check the detailed breakdown by region."
7. **Generation**: Returns $500M Q3, $450M Q2, 11% growth with supporting evidence

Agentic RAG vs. Traditional RAG

Aspect	Traditional RAG	Agentic RAG
Retrieval	One-shot, static	Multiple rounds, adaptive
Decision-making	None	Agent reasons about what to retrieve
Query decomposition	No	Agent breaks complex questions into sub-queries
Source selection	All sources equally	Agent picks specific relevant sources
Validation	No	Agent validates retrieved facts
Cost	Lower	Higher (more API calls)
Quality on complex tasks	Lower	Much higher
Latency	Lower	Higher

Knowledge Graphs in Agentic RAG

Knowledge graphs enable structured reasoning:

- **Entities**: People, products, organizations, concepts
- **Relationships**: "CEO of", "supplier to", "located in"
- **Queries**: Agent can traverse relationships instead of keyword search

Example:

```
Query: "Who are the suppliers of our manufacturing partners in Asia?"

Agent reasoning:
  1. Find our manufacturing partners in Asia
  2. For each partner, find their suppliers
  3. Filter to those in Asia
  4. Return results

KG traversal:
  Entity("Our Company")
    --[partnered_with]--> {manufacturing partners}
    --[in_region]--> {Asia}
    --[supplied_by]--> {suppliers}
```

RAG Best Practices

Data Preparation

- Clean and standardize documents
- Add metadata (source, date, author, category)
- Create embeddings offline for performance
- Index frequently accessed data

Retrieval Optimization

- Use hybrid search (keyword + semantic) for better coverage
- Implement filtering by metadata before semantic search
- Rerank results using more sophisticated models
- Set appropriate chunk size (usually 256-1024 tokens)

Integration with Agentic Loop

- Make retrieval a first-class action in the agent's tool set
- Track what was retrieved and from where (audit trail)
- Implement confidence scoring on retrieved facts
- Allow agent to request re-retrieval with different queries

Evaluation

- Measure retrieval quality (precision, recall)

- Track which sources the agent uses most
- Monitor hallucination rates
- A/B test different retrieval strategies

Agentic Design Patterns

Design patterns are reusable architectural solutions for common problems. Think of them as proven recipes for building reliable agentic systems.

Pattern 1: Tool Use (Core Foundation)

What it is: The LLM is given access to a menu of tools and decides when and how to use them.

When to use: Always. This is the foundational pattern.

Mechanism:

- Define tool schemas clearly
- LLM decides which tool to call based on the request
- Tool executes, returns result
- Loop back to LLM with observation

Example: E-commerce agent has tools like `search_products`, `get_user_cart`, `checkout`. For "show me wireless headphones", it calls `search_products(query="wireless headphones")`.

Pattern 2: Prompt Chaining (Sequential Steps)

What it is: Decompose a complex task into sequential steps where each LLM call's output feeds into the next call's input.

When to use: When you need guaranteed step order and predictable costs.

Example:

```
Step 1: Draft email
Step 2: Check for tone/grammar
Step 3: Optimize for brevity
Step 4: Get final approval
```

Pros: Predictable, cost-controlled, easy to debug **Cons**: Less flexible, can't adapt based on intermediate results

Pattern 3: Reflection (Self-Evaluation)

What it is: The agent generates an answer, then evaluates its own output and improves it.

When to use: When quality matters more than speed, or to reduce hallucinations.

Mechanism:

1. LLM generates initial response
2. Agent asks: "Is this answer correct? Complete? Well-reasoned?"
3. LLM reflects and refines
4. Repeat if needed

Example:

```
User: "Is Argentina bigger than Australia?"
1st attempt: "Yes, Argentina at 2.78M km² is larger than Australia at 7.69M
km²" ← Incorrect
Reflection: "Wait, let me recalculate: Australia 7.69M > Argentina 2.78M. I
made an error."
Corrected: "No, Australia (7.69M km²) is larger than Argentina (2.78M km²)"
```

Pattern 4: Planning (Think Before Acting)

What it is: The agent creates a step-by-step plan before taking action.

When to use: For complex multi-step tasks, when you want transparency, or to catch errors early.

Mechanism:

1. User provides goal
2. Planner LLM creates detailed plan with sub-goals
3. Executor LLM carries out each step
4. Monitor evaluates progress and adjusts if needed

Example:

```
User: "Plan a 3-day trip to Japan"

Plan:
1. Choose cities (Tokyo, Kyoto, Osaka)
2. Find flights to Tokyo
3. Book hotels in each city
4. Plan activities and restaurant reservations
5. Arrange transport between cities

Execution: Do each step in order
```

Pattern 5: ReAct (Reasoning + Acting)

What it is: Interleave reasoning with action. The agent thinks about what to do, acts, observes results, and reasons about what to do next.

When to use: Most practical applications combine elements of ReAct.

Format:

```
Thought: I need to find information about...
Action: search_web(query="...")
Observation: The search returned...
Thought: This is relevant because... I should now...
Action: retrieve_document(id="...")
Observation: The document says...
Thought: Now I have enough information to answer the user's question.
Final Answer: ...
```

Pattern 6: Multi-Agent Collaboration

What it is: Multiple specialized agents work together, each with different roles and tools.

When to use: When tasks span different domains or require different expertise.

Example: Content Creation Team

- **Researcher Agent**: Finds sources and facts
- **Writer Agent**: Drafts content based on research
- **Editor Agent**: Reviews for grammar and clarity
- **Marketing Agent**: Optimizes for SEO

Each agent focuses on what it does best, passing results to the next.

Challenges:

- Coordination overhead
- Debugging (which agent caused the problem?)
- Cost (multiple LLM calls)
- Orchestration complexity

Pattern 7: Step Budget & Circuit Breaker (Safety)

What it is: Limit the number of steps an agent can take before stopping or requiring human approval.

When to use: Always in production. Critical for cost control and safety.

Implementation:

```
steps_taken = 0
max_steps = 10  # Prevent infinite loops

while steps_taken < max_steps:
    thought = llm.reason()
    action = llm.decide_action(thought)
    result = execute(action)
    steps_taken += 1

if steps_taken == max_steps:
    return "Max steps reached. Human review needed."
```

Pattern 8: Tool Hierarchy & Guardrails

What it is: Organize tools by risk level and restrict dangerous operations.

Risk Levels:

- **Read-only**: Search, retrieve, analyze (safe)
- **Create**: Add data, create records (medium)
- **Modify**: Update existing data (medium-high)
- **Delete**: Remove data (high)
- **Financial**: Transfer money, make purchases (critical)

Guardrails:

- Require human approval for risky operations

- Implement rate limiting
- Log all actions
- Validate parameters strictly
- Use financial transaction limits

When to Use Which Pattern

Task Type	Recommended Patterns
Simple lookup	Tool Use
Multi-step with fixed order	Prompt Chaining
Complex reasoning	ReAct + Reflection
Quality critical	Reflection, Planning
Domain-specific	Multi-Agent with specialists
Unknown/evolving	Tool Use + Reflection
Production (any)	+ Step Budget + Guardrails

Combining Patterns

Real production systems combine patterns:

```
Planning ("What's the best approach?")
  + Tool Use ("Call these APIs")
  + ReAct ("Reason about results")
  + Reflection ("Is the answer correct?")
  + Multi-Agent ("Specialist agents for specific domains")
  + Step Budget ("Stop after 10 steps")
  + Guardrails ("Require approval for risky actions")
```

Implementation: Building Your First Agent

Step-by-Step Implementation

Phase 1: Define the Problem

1. What task does the agent need to accomplish?

2. What information does it need to retrieve?
3. What actions must it take?
4. What constraints exist (budget, time, safety)?

Example: "Build a customer support agent that answers product questions using docs, searches the web if needed, and escalates complex issues to humans."

Phase 2: Design Tool Set

1. List all tools the agent will need
2. Define clear schemas for each tool
3. Implement error handling
4. Add rate limiting and guards

Example tools:

- `search_product_docs(query)` → Returns relevant product documentation
- `search_web(query)` → Searches the web for information
- `escalate_to_human(reason, customer_id)` → Transfers to support team

Phase 3: Build the Reasoning Layer

1. Write the system prompt defining the agent's role, constraints, and decision-making
2. Choose the LLM (Claude, GPT-4, Gemini, etc.)
3. Set up tool schemas in the LLM's format
4. Test basic tool calling

System prompt example:

```
You are a helpful customer support AI agent. Your job is to answer product
questions.
You have access to the following tools:
- search_product_docs: Find information in our product documentation
- search_web: Search the web for information
- escalate_to_human: Transfer complex issues to human support

Follow these rules:
1. Always try the product docs first
2. Only search the web if docs don't have the answer
3. Escalate issues involving account access, payment, or security
4. Be honest if you don't know something
```

Phase 4: Implement the Orchestration Loop

1. Create a loop that:
   - Takes user input
   - Calls the LLM with context
   - Parses tool calls from the response
   - Executes the tools
   - Feeds results back to the LLM
   - Repeats until done
2. Implement error handling:
   - Tool execution failures
   - Invalid function calls
   - Rate limit exceeded
   - Context window full
3. Add safety constraints:
   - Step counter (max 10 steps)
   - Timeout per step (30 seconds)
   - Request validation

Phase 5: Add Memory

1. Short-term: Pass conversation history in the prompt
2. Long-term: Optional - vector DB for knowledge storage
3. Episodic: Log agent actions and outcomes

Phase 6: Integrate Data Sources

1. Connect to live APIs, databases, knowledge bases
2. Implement caching for frequently accessed data
3. Handle data freshness and staleness
4. Add source attribution for compliance

Phase 7: Test & Iterate

1. Unit test each tool independently
2. Integration test the full loop
3. User testing with real queries
4. Identify failure modes

5. Improve prompts and tool design based on failures

Common Pitfalls & Solutions

Pitfall 1: Over-Engineering

Problem: Building a full multi-agent system with complex orchestration when a simple tool-using agent would work.

Solution: Start simple. Add complexity only when needed:

- Start: Single agent + simple tools
- Then: Add agentic RAG if single retrieval fails
- Then: Add multi-agent if domains are very different
- Then: Add planning/reflection if quality is low

Pitfall 2: Poor Tool Schemas

Problem: Tools with vague parameters, no constraints, or unhelpful descriptions. LLM hallucinates parameters.

Solution:

- Be explicit: "country_code must be a 2-letter ISO 3166 code, e.g., 'US', 'CA'"
- Add examples
- Restrict to enums when possible
- Validate at runtime

Pitfall 3: Ignoring Edge Cases

Problem: Agent works on happy path but fails on errors, unexpected inputs, or boundary conditions.

Solution:

- Error injection testing: What if an API is down?
- Timeout testing: What if an API is slow?
- Invalid input testing: What if the user inputs garbage?
- Edge case testing: Empty results, very large results, ambiguous queries

Pitfall 4: Cost Explosion

Problem: Agent makes hundreds of LLM calls, costing $100s.

Solution:

- Implement step budgets (max 10 steps)
- Cache tool results
- Use cheaper models for simple tasks
- Monitor token usage
- Set spending alerts

Pitfall 5: Hallucination

Problem: Agent confidently makes up information not retrieved from data sources.

Solution:

- Add reflection pattern (ask LLM to verify its answer)
- Require facts to come from retrieved data
- Implement confidence scoring
- Use guardrails to block made-up claims
- Add human review for critical tasks

Pitfall 6: Slow Response Time

Problem: Agent takes 10+ seconds to answer simple questions.

Solution:

- Parallelize tool calls
- Cache frequently accessed data
- Use streaming for real-time feedback
- Implement timeouts on tools
- Use faster models for simple tasks

Best Practices

Observability & Monitoring

- Log every tool call and its result
- Track success/failure rates per tool
- Monitor agent decision quality
- Alert on anomalies (cost spikes, error rates)

Prompt Optimization

- Keep system prompts concise and clear

- Use concrete examples
- Tell the agent what to do, not what not to do
- Update prompts based on failure analysis

Tool Design

- Make tools atomic and focused
- Return structured, parseable results
- Include helpful error messages
- Version tools when interfaces change

Testing

- Unit test tools independently
- Integration test the full loop
- Test edge cases and error conditions
- Collect real-world examples and add to test suite

Version Control

- Track system prompts as code
- Version tool schemas
- Keep logs of prompt changes and their impact
- Automate A/B testing of prompt variations

Frameworks & Tools for Building Agents

LangChain / LangGraph

What it is: Open-source framework for composing applications with LLMs.

Strengths:

- Modular: Agents, chains, memory, tools are separate components
- Wide integration: 200+ tool integrations pre-built
- Flexible: Can build simple chains or complex multi-agent systems
- Debugging: LangSmith provides logging and monitoring
- Large ecosystem: Community tools and extensions

Weaknesses:

- Learning curve: Lots of abstractions to understand
- Overhead: Sometimes slower than direct API calls
- Documentation: Can be hard to find exact patterns

Best for: Developers building production agents, those needing tool integrations

CrewAI

What it is: High-level framework for building multi-agent teams.

Strengths:

- Designed for multi-agent systems (role-based agents)
- Built-in communication patterns
- Task management
- Memory and knowledge sharing
- Human-in-the-loop capabilities

Weaknesses:

- Less flexible than LangChain
- Smaller ecosystem
- Newer, less battle-tested

Best for: Multi-agent team applications, role-based agents, structured workflows

OpenAI Swarm

What it is: Lightweight framework for agent coordination.

Strengths:

- Simple and minimal (good for learning)
- Designed for tool use
- Easy handoffs between agents
- Clean example code

Weaknesses:

- Limited features compared to LangChain
- Smaller community
- Less mature

Best for: Learning agents, simple tool-using agents

AutoGPT / BabyAGI

What it is: Full agent frameworks with autonomous task management.

Strengths:

- Can run fully autonomously
- Self-directed task planning
- Goal-oriented behavior

Weaknesses:

- High token costs
- Difficult to control and predict
- Overkill for most applications
- Can enter infinite loops

Best for: Experimentation, research, fully autonomous agents

Anthropic Claude with Tool Use

What it is: Direct use of Claude's native function calling capability.

Strengths:

- Minimal abstraction, maximum control
- Native tool calling is very reliable
- Lower latency and costs
- Excellent documentation
- Works with any LLM supporting tool calling

Weaknesses:

- You build orchestration yourself
- Less pre-built integrations
- More code to write

Best for: Teams wanting full control, custom applications, high-reliability systems

Framework Comparison

Framework	Complexity	Multi-Agent	Tool Integration	Community	Learning Curve
LangChain	Medium	Good	Extensive	Large	Medium
LangGraph	Medium-High	Excellent	Extensive	Growing	Medium-High
CrewAI	Medium	Excellent	Good	Small	Medium
Claude Direct	Low	Manual	Manual	N/A	Low
OpenAI Swarm	Low	Basic	Manual	Small	Low
AutoGPT	High	N/A	Extensive	Medium	High

Choosing a Framework

Start with Claude Direct if:

- You want full control
- You're building a one-off agent
- You want minimal dependencies
- You want to understand how agents work

Choose LangChain if:

- You need tool integrations with external systems
- You want a mature, proven framework
- You need debugging and monitoring
- You're building production applications

Choose LangGraph if:

- You need sophisticated state management
- You're building complex workflows
- You want visualization of agent behavior

Choose CrewAI if:

- You need multiple specialized agents
- You want role-based agents
- You need agent communication and knowledge sharing

Choose OpenAI Swarm if:

- You're learning about agents
- You want simplicity over features
- You want clean, readable code

Production Deployment & Scaling

Pre-Production Checklist

Reliability

- [ ] Agent completes tasks successfully >95% of the time
- [ ] Failures degrade gracefully (fallback behavior)
- [ ] All tools have timeout handling
- [ ] Errors are logged with sufficient context for debugging
- [ ] Step budgets prevent infinite loops
- [ ] Rate limiting prevents API overload

Safety

- [ ] No dangerous operations without approval
- [ ] Tool permissions properly scoped
- [ ] Input validation prevents injection attacks
- [ ] Output validation catches hallucinations
- [ ] Audit logging tracks all actions

Performance

- [ ] Response time meets SLA (<5 sec for most queries)
- [ ] Throughput meets expected load
- [ ] Memory usage is reasonable
- [ ] Token usage is optimized
- [ ] Caching reduces redundant API calls

Cost

- [ ] Token cost per task understood
- [ ] Cost scales reasonably with volume
- [ ] Tool usage is optimized
- [ ] Budget alerts configured

Monitoring

- [ ] Logging captures all relevant events
- [ ] Metrics tracked: success rate, latency, cost, errors
- [ ] Dashboards show agent health
- [ ] Alerts for anomalies

Deployment Architecture

Basic Setup

```
Client → API Gateway → Agent Service → LLM API
                            ↔ Tools/APIs
                            ↔ Databases
                            ↔ Vector DB
```

Production Setup

```
Clients (Web, Mobile, API)
    ↓
Load Balancer
    ↓
Agent Service (Kubernetes cluster, auto-scaled)
    ↓
Request Queue (async for batch jobs)
    ↓
Agent Workers
    ↑→↑ Cache (Redis)
    ↑→↑ Memory Store (Vector DB)
    ↑→↑ External APIs
    ↑→↑ Databases
    ↑→↑ LLM APIs
    ↓
Monitoring (Logs, Metrics, Traces)
    ↓
Alerts & Dashboards
```

Key Production Considerations

Monitoring & Observability

1. **Logging**

- Log every LLM call (prompt, response, tokens)
- Log every tool execution (input, output, latency, errors)
- Log agent decisions and reasoning
- Include correlation IDs for request tracing

2. **Metrics**
   - Success/failure rates by task type
   - Response time distribution
   - Tool usage frequency
   - Token usage per task
   - Error rates by type
   - Cost per task
3. **Distributed Tracing**
   - Track request flow from user to all downstream services
   - Identify bottlenecks
   - Understand failure paths
4. **Dashboards & Alerts**
   - Real-time health dashboard
   - SLA compliance tracking
   - Cost trending
   - Anomaly detection
   - Page on-call team for critical issues

Cost Optimization

1. **Token Management**
   - Measure tokens per task
   - Use cheaper models when possible
   - Cache results to reduce redundant calls
   - Compress prompts
   - Use batching for batch jobs
2. **Tool Optimization**
   - Cache API responses
   - Batch tool calls
   - Use search filters to reduce data retrieval

- Implement tool fallbacks (faster, cheaper alternatives)

3. **Budgeting**
   - Set monthly budget per agent
   - Cost alerts at 50%, 75%, 90%, 100%
   - Throttle or disable if over budget
   - Cost analysis by task type

Scaling

1. **Horizontal Scaling**
   - Run multiple agent instances
   - Load balance across instances
   - Use message queue for requests
   - Scale workers independently
2. **Caching Strategy**
   - Cache LLM responses for identical queries
   - Cache tool results (with TTL)
   - Cache embeddings for RAG
   - Use Redis or similar for fast access
3. **Async Processing**
   - Long-running agents should be async
   - Return job ID immediately
   - Provide polling or webhook for results
   - Use job queue for reliability

Security

1. **API Security**
   - Authentication & authorization
   - Rate limiting per user
   - Request validation
   - CORS policies
2. **Data Security**
   - Encrypt data in transit (HTTPS)
   - Encrypt sensitive data at rest

- Audit who accesses what
- PII handling and compliance

3. **Model Security**
   - Keep API keys secure (environment variables, secrets manager)
   - Monitor for unusual model behavior
   - Version control prompts
   - Test for adversarial inputs

Incident Response

1. **Common Issues**
   - Agent enters infinite loop → Step budget saves you
   - Tool fails → Fallback tool or graceful degradation
   - LLM API down → Queue requests, retry with exponential backoff
   - Data stale → Refresh cache, re-retrieve
2. **Debugging**
   - Check logs for exact failure point
   - Replay the request with saved context
   - Test the tool independently
   - Test the prompt against fresh data

A/B Testing & Continuous Improvement

What to A/B Test

- System prompts (different instructions, examples)
- Tool sets (add/remove tools, reorder)
- Data sources (different retrieval strategies)
- LLM models (faster/cheaper vs. smarter)
- Patterns (tool use vs. planning vs. reflection)

Metrics to Track

- Success rate (did the agent complete the task?)
- Correctness (was the answer right?)
- User satisfaction
- Cost (tokens used)

- Latency
- Tool usage patterns

Process

1. Define hypothesis ("Reflection pattern will improve correctness")
2. Implement variant
3. Route small % of traffic to variant
4. Collect metrics for statistical significance
5. Analyze results
6. Roll out winner or declare no difference
7. Iterate

Lifecycle Management

Versioning

- Version system prompts
- Version tool schemas
- Version models (e.g., Claude 3.5 Sonnet vs. 4.0)
- Track what changed between versions

Backwards Compatibility

- Support multiple versions during transition
- Gradual rollout of changes
- Ability to rollback quickly

Deprecation

- Plan deprecation timeline for old versions
- Notify users of changes
- Provide migration path
- Set sunset date